

Bibbidiba 開發任務 Phase B | 系統邏輯與演出

執行前請先閱讀 **AGENTS.md** 的全域開發規範 **Phase B** 必須在 **Phase A** 完成並確認無誤後才開始執行 本文件涉及檔案：`js/engine.js`、`js/ui.js`、`js/data.js`、`css/style.css`、`js/locales/zh-tw.js`、`js/locales/zh-cn.js`、`js/locales/en.js`、`js/locales/ja.js` 每個任務完成後請更新 `CHANGELOG.md`

任務一 | 傷害資訊可見性統一機制

目標：建立單一查詢來源，確保所有傷害相關 UI 在枷鎖生效時一律同步隱藏，不會有部分洩漏。

執行步驟

Step 1 — 閱讀現況

- 閱讀 `js/data.js`，找出所有與「隱藏傷害資訊」或「隱藏 HP 資訊」相關的現有枷鎖
- 閱讀 `js/engine.js`，確認枷鎖狀態的儲存方式與查詢方式
- 閱讀 `js/ui.js`，確認目前傷害相關 UI 各自如何判斷顯示條件
-  列出清單向製作人確認後再動工，嚴禁猜測

Step 2 — 新增統一查詢 API (`js/engine.js`)

新增以下查詢函式，作為所有傷害相關 UI 的唯一判斷來源：

```
// 預估傷害數字是否可見
isDamageVisible()
// → 當有枷鎖要求隱藏預估傷害數字時回傳 false

// 敵人血條是否可見
isEnemyHpBarVisible()
// → 當有枷鎖要求隱藏血條時回傳 false

// 血條預判閃爍條是否可見（新增，專管預判條）
isEnemyHpBarPreviewVisible()
// → 當有枷鎖要求隱藏血條預判條時回傳 false

// 取得當前應顯示的預估傷害值（供 ui.js 統一取值）
getDisplayedEstimatedDamage()
```

// → 預設回傳實際預估傷害值
// → 若「酒醉」枷鎖生效，回傳帶浮動的顯示值（見任務二）

Step 3 — 統一 UI 查詢（js/ui.js）

找到以下所有傷害相關 UI 渲染位置，統一改為查詢 engine API：

UI 元素	修改方式
預估傷害數字	改用 <code>engine.getDisplayedEstimatedDamage()</code> 取值
預估傷害金色高亮	<code>isDamageVisible()</code> 為 <code>false</code> 時，強制不顯示高亮
血條預判條	<code>isEnemyHpBarPreviewVisible()</code> 為 <code>false</code> 時隱藏
血條本體	<code>isEnemyHpBarVisible()</code> 為 <code>false</code> 時隱藏

原則：禁止在 `ui.js` 中自行判斷枷鎖狀態，一律呼叫 engine API。

任務二 | 新增枷鎖「煙霧」與「酒醉」

目標：新增兩個與傷害資訊顯示相關的枷鎖。

枷鎖設計說明

屬性	煙霧	酒醉
難度	輕度	重度
預估傷害數字	正常顯示	跳動浮動值
血條預判條	隱藏	顯示浮動寬度
金色高亮	正常判斷	依浮動值判斷
血條本體	正常顯示	正常顯示

執行步驟

Step 1 — 新增枷鎖資料（js/data.js）

⚠ 閱讀現有枷鎖資料結構，依照既有格式新增，欄位名稱以現有慣例為準

```
// 煙霧（輕度）
{
  id: 'shackle_smoke',
  difficulty: 'light',          // 以現有難度欄位格式為準
  type: 'info_hide',
  hidesDamagePreviewBar: true,
  hidesDamageNumber: false,
  distortsDamage: false
}

// 酒醉（重度）
{
  id: 'shackle_drunk',
  difficulty: 'heavy',          // 以現有難度欄位格式為準
  type: 'info_distort',
  hidesDamagePreviewBar: false,
  hidesDamageNumber: false,
  distortsDamage: true,
  distortRange: 0.20           // ±20% 浮動幅度
}
```

Step 2 — 酒醉浮動邏輯（js/engine.js）

擴充 `getDisplayedEstimatedDamage()`：

```
// 酒醉生效時：
// 每 300ms 重新計算浮動顯示值
displayValue = actualDamage × (1 + random(-0.20, 0.20))

// 計時器管理：
// - 進入含酒醉枷鎖的關卡時啟動 setInterval(300ms)
// - 離開關卡或枷鎖解除時 clearInterval
// - 計時器只更新內部快照值，禁止在 engine 內操作 DOM
// - 計時器觸發時，呼叫 ui.refreshDamageDisplay() 通知 ui 刷新
```

Step 3 — 酒醉視覺樣式（css/style.css）

```
@keyframes drunk-shake {
  0%   { transform: translateX(0px); filter: blur(0px); }
  25%  { transform: translateX(-2px); filter: blur(0.4px); }
  50%  { transform: translateX(2px); filter: blur(0.8px); }
  75%  { transform: translateX(-1px); filter: blur(0.4px); }
  100% { transform: translateX(0px); filter: blur(0px); }
}
```

```
.damage-drunk {
  animation: drunk-shake 0.3s ease-in-out infinite;
  color: #FFD700;
}

.damage-preview-bar.drunk {
  animation: dmg-preview-blink 0.3s ease-in-out infinite alternate;
}
```

- 酒醉枷鎖生效時，`ui.js` 在傷害數字與預判條 DOM 元素上加上 `.damage-drunk`
- 枷鎖解除時移除

Step 4 — 多語系（`js/locales/` 四個檔案）

同步新增以下 key（翻譯依對應語言）：

```
// zh-tw.js
shackle_smoke_name: '煙霧',
shackle_smoke_desc: '看不到血條預估傷害。',
shackle_drunk_name: '酒醉',
shackle_drunk_desc: '無法精準掌握預估傷害。',
```

完成後使用 `test_i18n.js` 或 `fix_i18n.js` 確認四個語系檔 key 值對齊。

測試情境

- 煙霧生效：數字正常 → 血條預判條消失 → 高亮正常
- 酒醉生效：數字每 300ms 在 $\pm 20\%$ 跳動 → 預判條寬度同步跳動 → `.damage-drunk` 動畫出現
- 兩者同時：數字跳動 + 預判條消失
- 枷鎖解除：計時器清除，所有顯示立即恢復正常

任務三 | 遺物逐步結算演出（Balatro 風格）

目標：攻擊時遺物區依序亮起並跳動顯示每個遺物的傷害加成，最後才打出最終傷害。

執行步驟

Step 1 — 閱讀現況

- 閱讀 `js/engine.js`：

- 找到傷害結算的核心函式
- 確認遺物加成的計算順序
- 確認目前是否保留每個遺物的中間值
- 確認攻擊觸發的完整流程
- 閱讀 `js/ui.js` :
 - 找到遺物區（A~D欄）的 DOM 結構
 - 確認攻擊演出的現有動畫流程
 - 確認輸入鎖定機制是否已存在
-  列出預計修改項目向製作人確認後再動工

Step 2 — engine 輸出結算步驟清單（`js/engine.js`）

新增 `calculateDamageSteps()` 函式，回傳結算步驟清單：

```
// 回傳格式範例：
[
  {
    relicId: 'relic_thunder_claw',
    relicName: '雷爪獅的祝福',
    type: 'multiply',           // 'multiply' | 'add'
    multiplier: 3.0,           // type 為 multiply 時
    bonus: null,               // type 為 add 時
    damageAfter: 300           // 套用此遺物後的傷害值
  },
  {
    relicId: 'relic_scale',
    relicName: '天秤之極',
    type: 'add',
    multiplier: null,
    bonus: 200,
    damageAfter: 500
  },
  // ...
  { final: true, damageAfter: 6495390 }
]
```

 結算步驟清單僅供演出使用，不影響實際傷害數值的計算邏輯 最終傷害必須與原本計算結果完全一致，禁止因此產生數值誤差

Step 3 — 逐步播放動畫（`js/ui.js`）

新增 `playDamageStepsAnimation(steps)` 函式：

播放流程（每一步）：

1. 對應遺物 DOM 元素加上 `.relic-active`（亮起效果）
2. 傷害數字以 `count-up` 動畫滾到本步驟的 `damageAfter`（約 150ms）
3. 若為 `multiply`：數字短暫放大縮回，閃爍金色，加上 `.damage-multiply`
4. 若為 `add`：數字上方浮出 `.damage-bonus-popup` 顯示 `+bonus` 值
5. 等待播放間隔後，遺物 `.relic-active` 移除，進入下一步
6. 全部播完後觸發原有攻擊演出動畫

播放間隔：

- 從設定中讀取「演出速度」，預設 400ms/步
- 若設定已有快/慢選項，沿用現有格式新增此選項

播放期間：

- 鎖定攻擊按鈕，禁止重複觸發
- 鎖定解除時機：攻擊演出動畫完全結束後

枷鎖處理：

- 攻擊演出開始後，無論煙霧/酒醉枷鎖是否生效，一律顯示 `steps` 清單中的真實 `damageAfter` 數值
- 不套用 `getDisplayedEstimatedDamage()` 的浮動邏輯
- 原因：攻擊已確認，傷害已定，枷鎖資訊遮蔽目的已達成

Step 4 — 演出樣式（`css/style.css`）

```
/* 遺物亮起 */
.relic-active {
  box-shadow: 0 0 12px 4px rgba(255, 215, 0, 0.85);
  transform: scale(1.12);
  transition: transform 0.1s ease, box-shadow 0.1s ease;
}

/* 乘法閃爍 */
@keyframes damage-multiply-flash {
  0% { transform: scale(1.0); color: #FFD700; }
  40% { transform: scale(1.25); color: #FFF176; }
  100% { transform: scale(1.0); color: #FFD700; }
}
.damage-multiply {
  animation: damage-multiply-flash 0.25s ease-out;
}

/* 加法浮動小字 */
@keyframes bonus-float {
  0% { opacity: 1; transform: translateY(0); }
```

```
    100% { opacity: 0; transform: translateY(-24px); }  
}  
.damage-bonus-popup {  
  position: absolute;  
  color: #A8FF78;  
  font-size: 0.85em;  
  animation: bonus-float 0.4s ease-out forwards;  
  pointer-events: none;  
}
```

視覺風格需與現有遺物卡片設計一致，可依實際情況微調數值。

測試情境

- 正常攻擊：遺物逐步亮起 → 數字跳動 → 最終攻擊打出
- 無遺物時：無逐步演出，直接攻擊
- 煙霧枷鎖生效：遺物演出正常，血條預判條不出現（演出前）；演出開始後顯示真實值
- 酒醉枷鎖生效：演出開始後數字顯示真實結算值，不再跳動
- 演出途中不可重複觸發攻擊

完成檢查清單

- `isDamageVisible()` / `isEnemyHpBarVisible()` / `isEnemyHpBarPreviewVisible()` 建立完成
- `getDisplayedEstimatedDamage()` 統一取值，`ui.js` 全面改用
- 煙霧枷鎖：血條預判條隱藏，其他正常
- 酒醉枷鎖：傷害數字與預判條每 300ms 跳動，`.damage-drunk` 動畫套用
- 兩枷鎖同時生效行為正確
- 四個語系檔新增枷鎖名稱與描述，`test_i18n.js` 驗證通過
- 遺物逐步演出依序播放，數字正確跳動
- 演出期間攻擊按鈕鎖定
- 演出中不受煙霧/酒醉枷鎖干擾，顯示真實值
- `CHANGELOG.md` 已更新

